Dylan Thornburg & Ben Banh

Digital Signal Processing Laboratory ELEN

133L Spring 2025

April 24, 25, 2025

Laboratory 3: Detecting DTMF Tones with FIR Bandpass Filters

Prelab due Wednesday April 23 at 8 pm

Objective

One way to automatically interpret the seven-tone DTMF signal created by dialing a telephone number is to process the signal in parallel with seven frequency selective filters, each of which is designed to suppress all frequencies except a narrow band of frequencies close to one of the seven DTMF frequencies and compare the energy of each output over short time intervals. In this laboratory we will use MATLAB to explore the most appropriate filter length and filter parameters to distinguish the button frequencies.

Background

An ideal bandpass filter completely suppresses all frequencies except the frequencies in a frequency band specified by the low frequency and high frequency that define the band or by the band center frequency and the width of the band. To achieve the "ideal" IDTFT filter, an infinite number of filter coefficients would be needed. Reliable detection needs filters that are "good enough", but not "ideal." The MATLAB function *fir1* computes the coefficients for a filter of a specified finite length that is the best approximation (in a least squares sense) for an ideal bandpass filter specified by its band edge frequencies. The filter created will not be as good as the "ideal" filter, and the frequency response of the approximate filter can be computed using the MATLAB function *freqz*. This result can be used to determine if the filter length selected is long enough to meet the performance requirements of the filter.

These functions are described in the Prelab instructions.

Laboratory Procedure

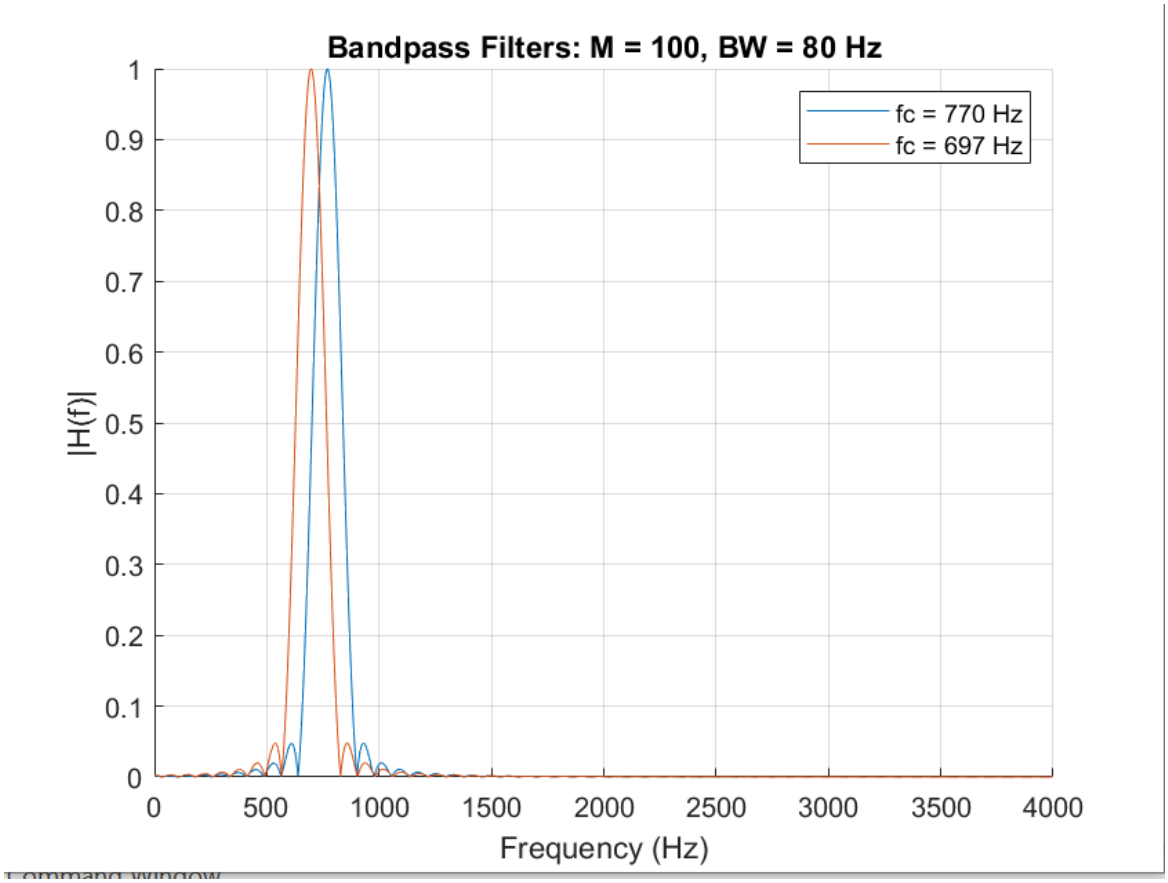
Step 1: Modify your livescript to be able to compare the performance of two filters for two different row frequencies.

- Modify Section 1 to specify center frequencies for two rows. Use M = 100, bw = 80, fc1 = 697, fc2 = 770 and fs = 8000.
- Modify Section 2 to compute two filters, b1 and b2. Compute the 1x2 vectors Wn1 and Wn2 for the two center frequencies using the same bw. Plot abs(H1) and abs(H2) on the same axes using a single plot instruction. Verify that your plot shows two similarly shaped narrow band filters, one centered at 697 Hz and one centered at 770 Hz. Include the values of M and bw in the plot title and show the fc values in a legend on the plot.
- Include this plot in your lab report.

CODE:

```
fs = 8000;
fc0 = 770;
fc1 = 697;
bw = 80;
M=100;

Wn0 = [fc0-bw, fc0+bw]/(fs/2);
Wn1 = [fc1-bw, fc1+bw]/(fs/2);

b0=fir1(M, Wn0, rectwin(M+1));
b1=fir1(M, Wn1, rectwin(M+1));


Np = 2048;
[H0, fv] = freqz(b0,1,Np,fs);
[H1, fv] = freqz(b1,1,Np,fs);
hold on
plot(fv, abs(H0))
plot(fv, abs(H1))
grid on
hold off
title(sprintf('Bandpass Filters: M = %d, BW = %d Hz', M, bw));
xlabel('Frequency (Hz)');
ylabel('|H(f)|');
legend(sprintf('fc = %d Hz', fc0), sprintf('fc = %d Hz', fc1));
```

PLOT:



Checkpoint 1: Show your plot to your laboratory assistant.

Step 2: Compare the filter impulse responses.

- Add Section 3 to plot b1 and b2 in two subplots.
- Run Section 3. How many positive peaks are seen in b1? In b2? How is this related to the respective center frequencies?

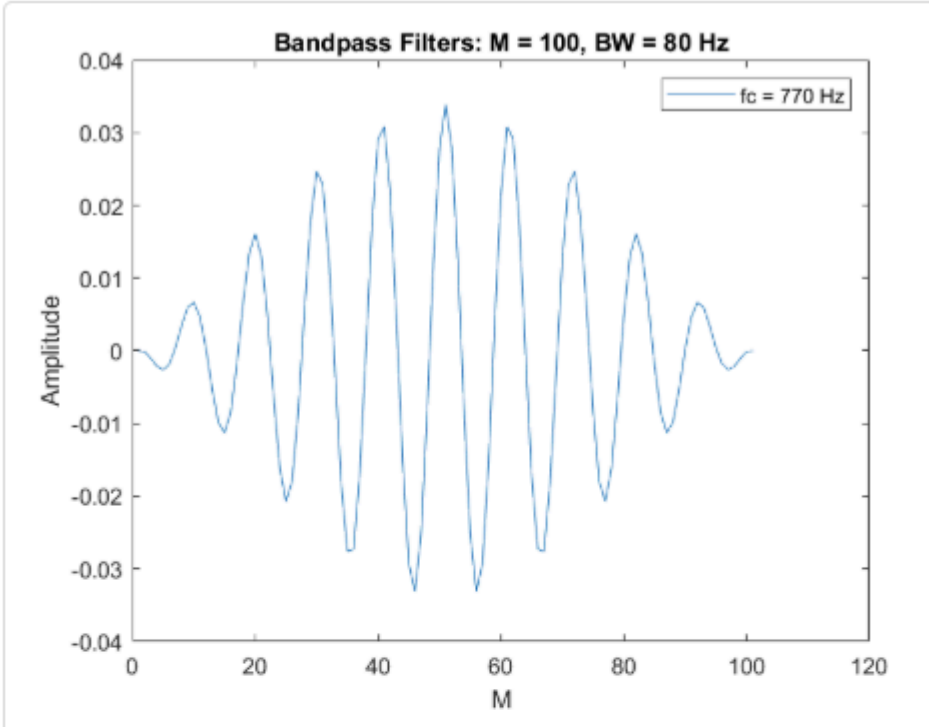
We see 9 positive peaks for b0 (770Hz) and also 9 for b1 (697Hz). The higher the center frequency, the more attenuations we will see. In this case, they are the same, but a frequency like 300Hz would have less peaks, and a frequency like 1300Hz would have more. We can even see that the right/left-most peaks for the 770Hz graph are higher than the 697Hz graph.

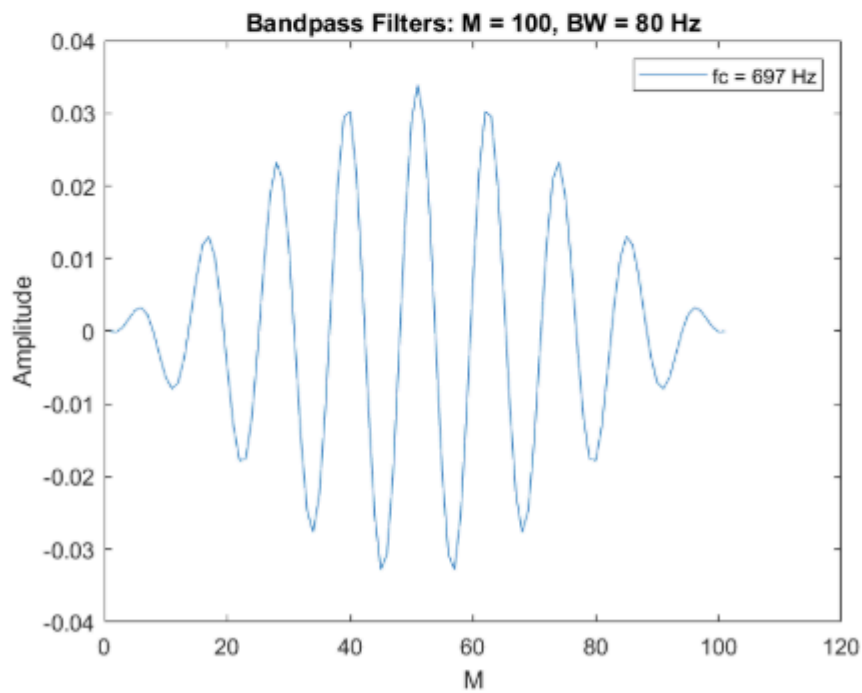
- What is the ratio of the value of the center peak in each filter to the peak values at the leftmost and rightmost peaks?

The ratio of the values of the center peak in f=697 Hz between the center and the left peak is roughly 10.

The ratio of the values of the center peak in f=770 Hz between the center and the left peak is roughly 5.5.

- Include this plot in your lab report.





Step 3: Create a DTMF signal using your function generator from Lab 2.

- Add Section 4 copied from Lab 2 which creates a test signal with fs = 8000, t1 = 0.1, t2 = 0.02, and dial_vals = [1 4 7 3 6 9]. Use plot and sound to check your signal.

- Run Section 4.

Step 4: Filter your test signal with your two filters.

• Create Section 5 to observe the effect of your filters on your test signal. Compute y1 and y2 using the filter function with filters b1 and b2, which were computed in Section 2.

• Use two subplots to plot y1 and y2.

• What is the maximum amplitude for each of the six buttons when filter b1 (770Hz) is used?

1:0.22
2:0.49
3:0.18
4:0.24
5:0.50
6:0.18

When filter b2 (697Hz) is used?

1: 0.50
2: 0.23
3: 0.023
4: 0.50
5: 0.23
6: 0.023

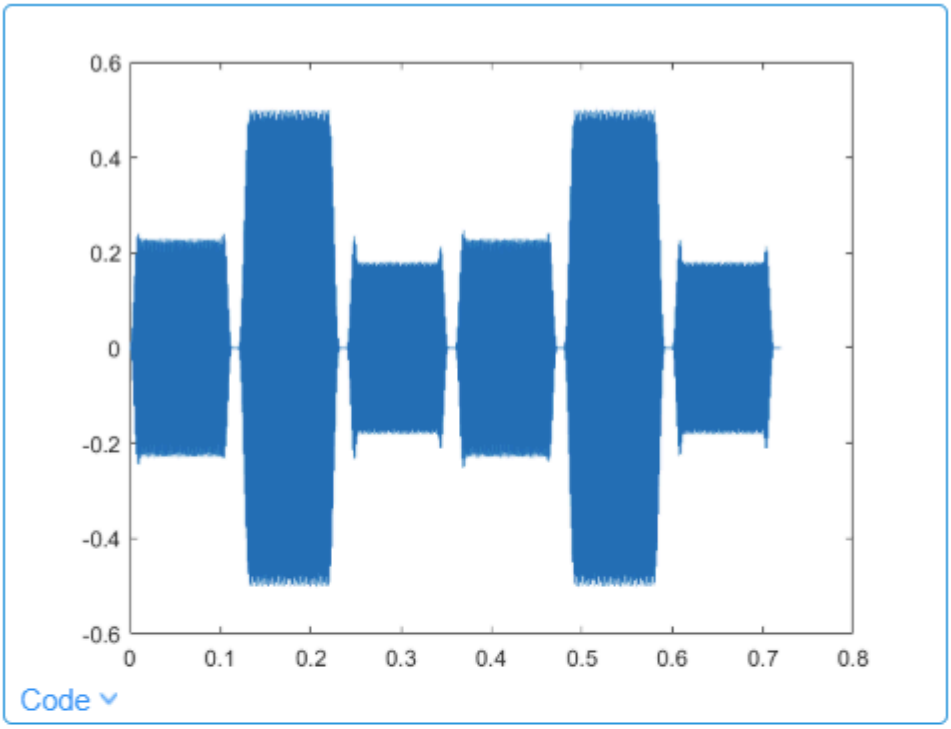
How do you explain the differences?

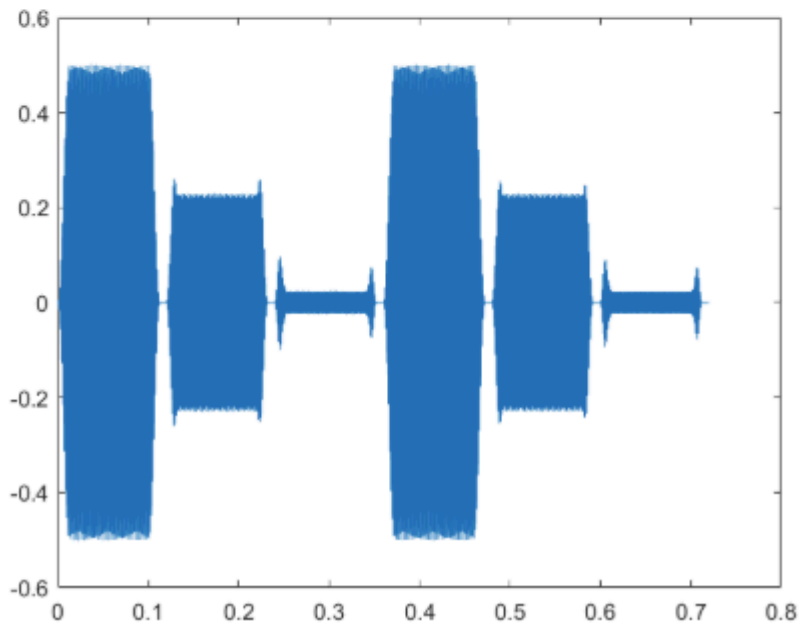
For filter b2, we want to only allow 697Hz through, whereas for filter b1 we want 770 to go through. Based on our bw of 80, 770 allows the third row freq of 852 to go through way more (which we also saw in the prelab).

Is there any noticeable dependence of the amplitudes on which column frequency was present for the six buttons?

The amplitudes filtered out the combined signals and let only part of the signal through. The row frequency was present while the column frequency was attenuated through the filter.

• Include this plot in your lab report.





Top is 770Hz

CODE:

```
plot(b1)
title(sprintf('Bandpass Filters: M = %d, BW = %d Hz', M, bw));
xlabel('M');
```

```
ylabel('Amplitude');
legend(sprintf('fc = %d Hz', fc1));

plot(b2)
title(sprintf('Bandpass Filters: M = %d, BW = %d Hz', M, bw));
xlabel('M');
ylabel('Amplitude');
legend(sprintf('fc = %d Hz', fc2));

tone_len = .1;
quiet_len = .02;
Fs = 8000;
n = 6;
dT = 1/Fs;

x = my_dtmf(tone_len, quiet_len, Fs, [1, 4, 7, 3, 6, 9]);

time = n * (tone_len + quiet_len);

tvec = (0: (time * Fs -1) ) * dT;
plot(tvec, x)
sound(x, Fs)

a=1;
y1 = filter(b1,a,x);
plot(tvec,y1);
sound(y1, Fs)


y2 = filter(b2,a,x);
plot(tvec,y2);
sound(y2, Fs)
```

Checkpoint 2: Show your plot to your laboratory assistant, and explain the maximum levels you observe.

Step 5: Make a detector for the presence of the two row frequencies.

Although the results of Step 5 show differences in the response of the row frequencies, a detector that could automatically identify which button was pressed at any moment in time would need to look at seven indicator signals, one for each row and column frequency. The indicator function would have a value of 1 when the row or column frequency was strong and a value of 0 otherwise.

Create Section 6 to make indicator functions for the first two rows. The indicator functions will

be created by rectifying y1 and y2 and then determining a threshold value that produces a 1 whenever the rectified y signal was above the threshold.

- Create signals y1a and y2a which will be the absolute values of y1 and y2 respectively. Plot them in the top row of a 3x2 subplot.

- Create y1s and y2s by smoothing y1a and y2a with a moving average filter of length smoothLength. This can be done using the filter function, with a filter created by bma=(ones(1,smoothLength))/smoothLength. A length of 20 is suggested, but you can explore the effect of changing the length. How many cycles of a signal with frequency fc1 would there be in 20 samples?

  N * fc1/fT = 20*697/8000 = 1.743 cycles

- Plot y1s and y2s in the second row of the 3x2 subplot.

- Observe the plots for y1s and y2s, and determine the lowest value you could select for a threshold to always have a 1 for the corresponding row frequency and never have a 1 for any of the other 6 frequencies. Record this in your lab report.

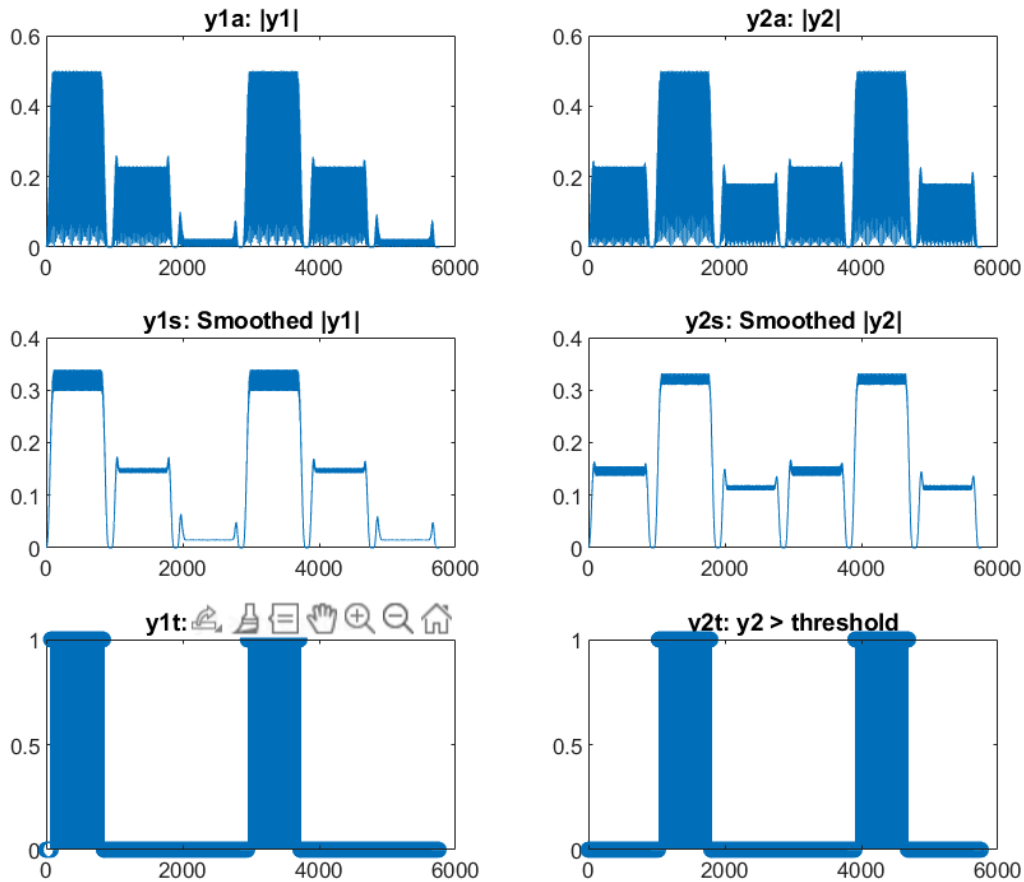
  Min = 0.17(highest value produced by input signal that we don't want)

- Determine the highest value for a threshold that would never give a zero result when the corresponding frequency was present. Record this in your lab report.

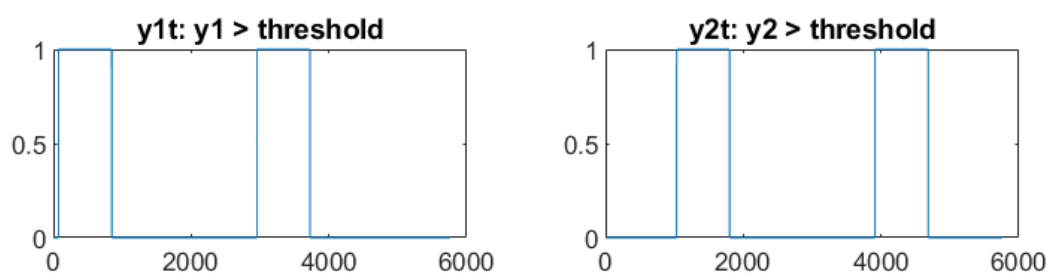
Max= 0.3 (lowest val for the signal we do want through from the smoothed signal)

- Set a threshold value to the average of the high and low values you just observed, and use it to create logical functions y1t and y2t which will be 1 when the signal is above the threshold and 0 when it is below. Plot these two signals in the bottom row of the 3x2 subplot.

- Include the 3x2 subplot in your lab report.



NO STEM:



CODE:

```
y1a = abs(y1);
y2a = abs(y2);

smoothLength = 20;                % Length of moving average filter
bma = ones(1, smoothLength) / smoothLength;  % Moving average filter


y1s = filter(bma, a, y1a);
y2s = filter(bma, a, y2a);
```

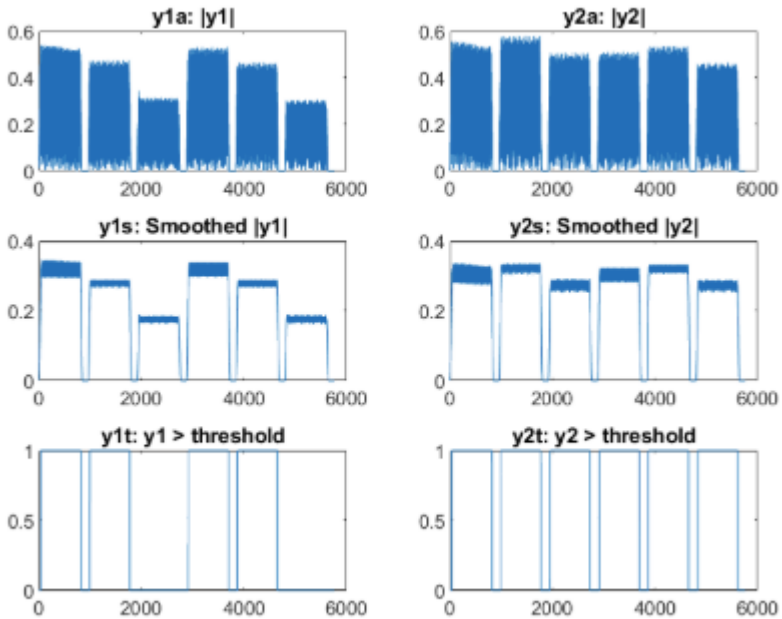
```
figure;

% Top row: Absolute values
subplot(3,2,1); plot(y1a); title('y1a: |y1|');
subplot(3,2,2); plot(y2a); title('y2a: |y2|');

% Middle row: Smoothed absolute values
subplot(3,2,3); plot(y1s); title('y1s: Smoothed |y1|');
subplot(3,2,4); plot(y2s); title('y2s: Smoothed |y2|');


lowThresh1 = .17;
highThresh1 = .3;
lowThresh2 = .17;
highThresh2 = .3;
thresh1 = (lowThresh1 + highThresh1) / 2;
thresh2 = (lowThresh2 + highThresh2) / 2;
y1t = y1s > thresh1;
y2t = y2s > thresh2;

% Bottom row: Thresholded logical signals
subplot(3,2,5); stem(y1t); title('y1t: y1 > threshold');
subplot(3,2,6); stem(y2t); title('y2t: y2 > threshold');
```

Checkpoint 3: Show your plot to your laboratory assistant, and explain your results.

Step 6: Explore the effect of filter parameters on the reliability of detection. The length of the filter determines the amount of computation required to produce the filter output, so if the filter length can be reduced, the implementation will be more efficient. However, longer filters are better able to distinguish frequencies that are close together or to make a narrower passband to suppress noise, so making a filter too short will reduce performance.

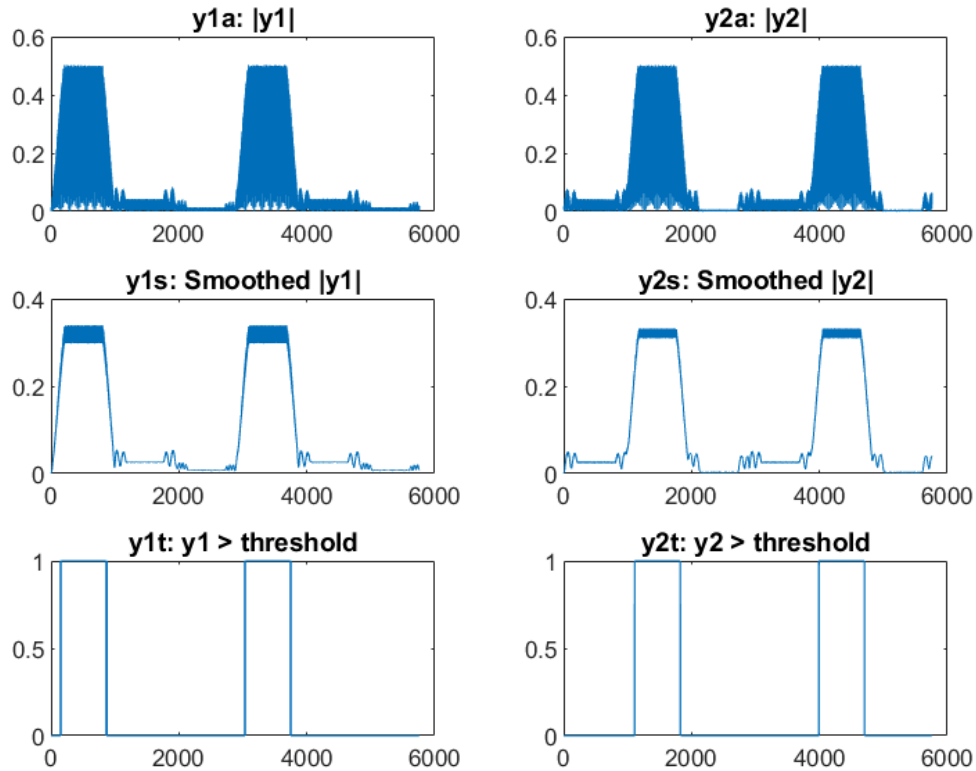
- In Section 1, change M to 26 to make a shorter filter and leave the other four parameters unchanged. Rerun the Sections in Steps 1,2,4, and 5 with this shorter filter. Use the same test signal from Step 3.
    - Compare the low and high threshold limits of the results of this shorter filter to the results with M = 100.

    Previous signals are now allowed through the filter:



- In Section 1, change M to 200 to make a longer filter, change bw to 20 to make a narrower filter, and leave the other three parameters unchanged. Redo Steps 1,2,4, and 5 with this longer and narrower filter. Use the same test signal from Step 3. Compare the results of this longer and narrower filter to the results with M = 100.
    - Compare the plot in Section 2 to the results from your prelab where bw was 20 but M was 100. Did increasing the filter length to 200 result in a narrower filter?
      Yes it did. It also multiplies the undesired signals by a smaller amplitude.

    - Compare the low and high threshold limits of this longer filter to the results with M = 100 and bw = 80.
      Now the current threshold has a high of 0.3 and a low threshold of 0.05. This threshold low limit is lower than before and thus is a more efficient filter by filtering a larger range of frequencies.



Checkpoint 4: Show your results to your laboratory assistant.

Step 7: Explore the effect of a noisy signal. All systems that will operate effectively in the real world must be designed with some allowance for random variations in the signals.

- In the command window, type "help randn" to read about the function that creates a random signal.

- In a new Section 7, make a noisy signal by adding random noise to your DTMF test signal using: testsign = testsig + *aa**randn(1, length(testsig));
    - Scale the noisy signal to have a maximum amplitude of 1 and listen to it with sound.
    - Select a value for the parameter *aa* that results in audible noise when you listen to the signal.

- Rerun the sections in Steps 4 and 5 with testsign and determine what impact the noise has on the low and high threshold values.

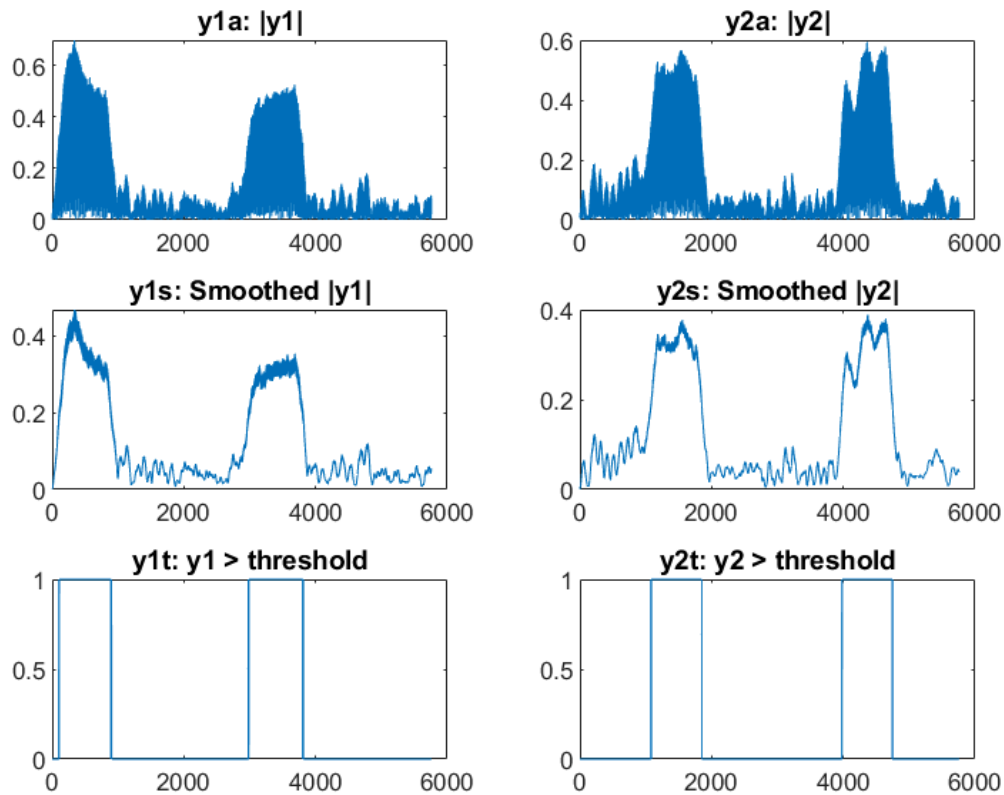
The noise makes the high and low thresholds hard to determine. It pushes them closer to the average amplitude of the signals. Ultimately, we settled on lows and highs of .23 and .15 which seem to always filter out the noise.

• Increase *aa* by a factor of 4 and determine what impact the noise has on the low and high threshold values.

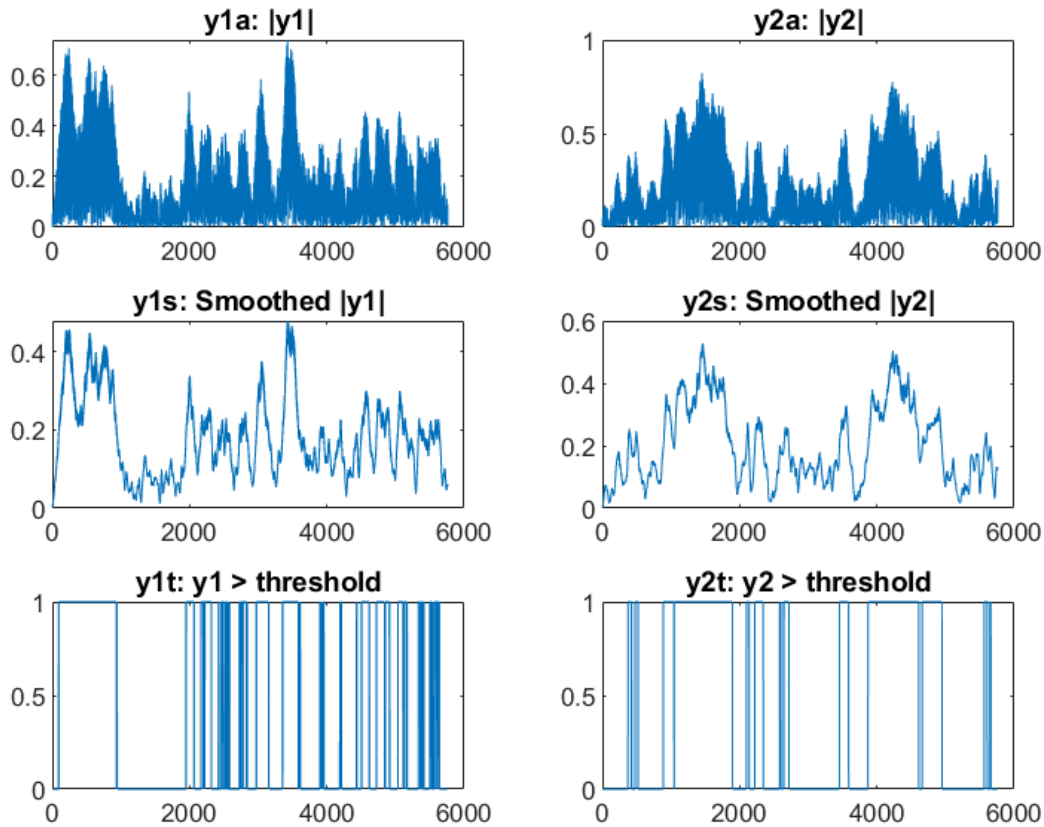
The added noise makes the threshold values so sporadic that determining proper threshold values is more difficult. For the values we had, the noise produced makes it so we can no longer distinguish the individual dial tones.

PLOTS:

aa = ½



aa = 2



CODE:

```
testsign = x + 1/2*randn(1, length(x));
sound(testsign)
a=1;

y1 = filter(b1,a,testsign);
y2 = filter(b2,a,testsign);

y1a = abs(y1);
y2a = abs(y2);

smoothLength = 20;                % Length of moving average filter
bma = ones(1, smoothLength) / smoothLength;  % Moving average filter


y1s = filter(bma, a, y1a);
y2s = filter(bma, a, y2a);

figure;

% Top row: Absolute values
```

```
subplot(3,2,1); plot(y1a); title('y1a: |y1|');
subplot(3,2,2); plot(y2a); title('y2a: |y2|');

% Middle row: Smoothed absolute values
subplot(3,2,3); plot(y1s); title('y1s: Smoothed |y1|');
subplot(3,2,4); plot(y2s); title('y2s: Smoothed |y2|');


lowThresh1 = .23;
highThresh1 = .15;
lowThresh2 = .23;
highThresh2 = .15;
thresh1 = (lowThresh1 + highThresh1) / 2;
thresh2 = (lowThresh2 + highThresh2) / 2;
y1t = y1s > thresh1;
y2t = y2s > thresh2;

% Bottom row: Thresholded logical signals
subplot(3,2,5); plot(y1t); title('y1t: y1 > threshold');
subplot(3,2,6); plot(y2t); title('y2t: y2 > threshold');
```

Checkpoint 5: Show your results to your laboratory assistant.

Laboratory report:

Submit the answers to the questions and the plots requested in the laboratory procedure.

List all the parameters used in making your filter/rectifier detector. What is the effect of changing M? bw? smoothLength?

Thresholds: Determines what signals are logic ‘1’ or ‘0’.

M: Refers to the order, or amount of past inputs that affect the current output (delay). By increasing M, we can make our filter longer (better), and by decreasing we can make our filter shorter (worse). However, increasing M comes at a huge computational cost, which is why we can’t just make M as big as we want.

Bw: Bandwidth of the signal. Increasing the bandwidth provides a larger acceptable signal to be passed through the filter, while a smaller bw is more strict about what signals can be passed through.

smoothLength: Increasing the smoothLength increases the moving average so the resulting output would be more smooth as it is averaged across more points.

aa(noise constant): determines the noise added to the input signal through the rand function.

Fs: the sampling frequency of the waveform.

In the design process for this lab, we tried to suppress nearby row and column frequencies by a large factor. However, your detector might still work reliably with less attenuation of nearby DTMF frequencies, and using a shorter filter would reduce both computation time and delay. Considering the tests you did in this lab, how much attenuation of nearby DTMF frequencies is needed for reliable operation in a noise free environment? What is the impact of added noise?

In a noise free environment, no attenuation is needed as signals are different enough that they are spaced apart on each frequency band. With added noise, attenuation is necessary as signals have the possibility of overlapping, making the detector unreliable.

(c) Copyright 2000-2025 Sally Wood